Taller de Programación Concurrente por Paso de Mensajes

Materia: Modelos y Paradigmas de Programación **Universidad del Valle Estudiante:** Raúl A. Alzate

---

Punto 1: Implementación de Puertos con Celdas (`PuertosConCeldas.oz`)

Descripción del problema

Implementar el concepto de puerto como un TAD con estado, empaquetado y seguro, utilizando celdas. Un puerto es un canal de comunicación con dos operaciones: `NuevoPuerto` (crea un puerto con flujo S y referencia P) y `Enviar` (envía un mensaje X por el puerto P).

Solución

Se utiliza una celda para almacenar la referencia al final del flujo (stream). Cada vez que se envía un mensaje, se realiza un `Exchange` atómico sobre la celda: se obtiene el final actual del flujo, se unifica con `Msg|NuevoFinal`, y la celda apunta al nuevo final.

```
None
proc {NuevoPuerto S P}
   C = {NewCell S}
in
   proc {P Msg}
      Old New
   in
      {Exchange C Old New}
      Old = Msg|New
   end
end
```

¿Por qué funciona?

- La celda `C` mantiene una referencia mutable al final no ligado del flujo `S`.
- `Exchange` es atómico: lee el valor actual y lo reemplaza en una sola operación, lo que garantiza seguridad ante accesos concurrentes.

- Al unificar `Old = Msg|New`, el mensaje queda añadido al flujo y `New` se convierte en el nuevo final libre.
- El procedimiento `P` actúa como el "puerto" encapsulado: solo a través de él se puede enviar mensajes.

Prueba

```
None
local S P in
   {NuevoPuerto S P}
   {Enviar P a}
   {Enviar P b}
   {Enviar P c}
   {Browse S}  % Resultado: a|b|c|_
end
```

Punto 2: Implementación de Celdas con Puertos (`CeldasConPuertos.oz`)

Descripción del problema

Implementar celdas (contenedores de valores mutables) usando objetos puerto. Las operaciones son: `NuevaCelda` (crea celda con valor inicial), `Acceder` (lee el valor) y `Asignar` (cambia el valor).

Solución

Se crea un puerto cuyo hilo servidor mantiene el estado actual de la celda. Los mensajes `acceder(R)` y `asignar(NuevoVal)` se envían al puerto, y el loop los procesa secuencialmente.

```
None
proc {NuevaCelda X C}
   Sin
   proc {MsgLoop S1 Estado}
      case S1 of Msg|S2 then
         case Msg
         of acceder(R) then
            R = Estado
            {MsgLoop S2 Estado}
         [] asignar(NuevoVal) then
            {MsgLoop S2 NuevoVal}
```

```
         end
      [] nil then skip
      end
   end
in
   {NewPort Sin C}
   thread {MsgLoop Sin X} end
end
```

¿Por qué funciona?

- El puerto recibe mensajes y los serializa en un flujo.
- El hilo `MsgLoop` procesa cada mensaje secuencialmente, garantizando consistencia del estado.
- `acceder(R)`: unifica `R` con el estado actual (el cliente espera hasta que R se ligue).
- `asignar(NuevoVal)`: la recursión continúa con `NuevoVal` como nuevo estado.

Prueba

```
None
local C V1 V2 in
   {NuevaCelda 10 C}
   {Acceder V1 C}   % V1 = 10
   {Asignar 25 C}
   {Acceder V2 C}   % V2 = 25
end
```

Punto 3: Contador con Múltiples Clientes (`contador.oz`)

Descripción del problema

Implementar un servidor contador que reciba caracteres de múltiples clientes concurrentes y genere un flujo de salida donde cada posición corresponde al total acumulado de ocurrencias hasta ese momento.

Solución

El servidor crea un puerto. Un hilo procesa el flujo de mensajes del puerto, acumulando las ocurrencias en una lista de pares `Char#Count`. Cada vez que llega un carácter, se actualiza el estado y se emite al flujo de salida.

```
None
fun {ContadorServidor}
   Sin
   P = {NewPort Sin}
   fun {Procesar S Estado}
      case S of Msg|Sr then
         NuevoEstado = {IncChar Msg Estado}
      in
         NuevoEstado|{Procesar Sr NuevoEstado}
      [] nil then nil
      end
   end
   Salida
in
   thread Salida = {Procesar Sin nil} end
   puerto(enviar:proc {$ X} {Send P X} end salida:Salida)
end
```

Análisis del no-determinismo

La salida mostrada en el enunciado:

```
None
[a#1] | [a#1 b#1] | [a#2 b#1] | [a#2 b#1 c#1] | [a#2 b#1 c#2] | ...
```

No es la única posible. La razón es que los tres clientes ejecutan en hilos concurrentes y el planificador (scheduler) puede intercalar los mensajes en cualquier orden. El puerto serializa los mensajes en el orden en que llegan, pero ese orden de llegada es no-determinista.

Lo único garantizado es el orden relativo dentro de cada cliente:

- Cliente 1: a siempre antes que c
- Cliente 2: b siempre antes que c
- Cliente 3: solo envía a

Otras posibles salidas:

Orden de llegada	**Flujo de salida**
b, a, c, a, c	`[b#1] | [a#1 b#1] | [a#1 b#1 c#1] | [a#2 b#1 c#1] | [a#2 b#1 c#2]`

Orden de llegada	Flujo de salida
a, c, b, c, a	`[a#1] | [a#1 c#1] | [a#1 b#1 c#1] | [a#1 b#1 c#2] | [a#2 b#1 c#2]`
a, a, b, c, c	`[a#1] | [a#2] | [a#2 b#1] | [a#2 b#1 c#1] | [a#2 b#1 c#2]`

Punto 4a: Portero de Concierto (`portero.oz`)

Descripción del problema

Implementar un objeto puerto `Portero` que cuente las personas en un concierto. Recibe mensajes: `getIn(N)` (entran N personas), `getOut(N)` (salen N personas), `getCount(N)` (liga N con el conteo actual).

Solución

Se usa la abstracción `NuevoObjetoPuerto` dada en el enunciado. La función `Comp` (`PorteroComp`) define el comportamiento:

```
None
fun {PorteroComp Msg Estado}
   case Msg
   of getIn(N) then Estado + N
   [] getOut(N) then Estado - N
   [] getCount(N) then
      N = Estado
      Estado
   end
end
```

Análisis:

- `getIn(N)`: retorna `Estado + N` como nuevo estado (no reactivo — N es un valor de entrada).
- `getOut(N)`: retorna `Estado - N` como nuevo estado (no reactivo).
- `getCount(N)`: **reactivo** — liga la variable `N` del mensaje con el estado actual. El cliente que envió `getCount(N)` espera hasta que `N` se unifique, obteniendo así la respuesta. El estado no cambia.

Prueba

```
None
local P N1 N2 in
   P = {Portero}
   {Send P getIn(5)}
   {Send P getIn(3)}
   {Send P getCount(N1)}   % N1 = 8
   {Send P getOut(2)}
   {Send P getCount(N2)}   % N2 = 6
end
```

Punto 4b: Contador con NuevoObjetoPuerto (`contador2.oz`)

Descripción del problema

Reimplementar el servidor Contador usando `NuevoObjetoPuerto`. La función `{Counter Output}` retorna un objeto puerto. `Output` es un flujo de salida con las listas de conteos acumulados en cada momento.

Solución

La clave (pista del enunciado) es usar **un segundo puerto** para el flujo de salida. La función `Comp` del objeto puerto, al procesar cada carácter, envía el nuevo estado al puerto de salida y retorna el nuevo estado.

```
None
fun {Counter Output}
   SalidaSin
   PSalida = {NewPort SalidaSin}
   fun {ContadorComp Msg Estado}
      NuevoEstado = {IncChar Msg Estado}
   in
      {Send PSalida NuevoEstado}
      NuevoEstado
   end
in
   Output = SalidaSin
   {NuevoObjetoPuerto ContadorComp nil}
end
```

- `PSalida` es un puerto interno cuyo flujo `SalidaSin` se unifica con `Output`.

- Cada vez que llega un mensaje al objeto puerto, `ContadorComp` actualiza el conteo, envía el estado acumulado al puerto de salida, y retorna el nuevo estado.
- El resultado es que `Output` es un flujo creciente de listas de pares `Char#Count`.

Prueba

None

```
local Output Servidor in
   Servidor = {Counter Output}
   thread {Send Servidor a} {Send Servidor c} end
   thread {Send Servidor b} {Send Servidor c} end
   thread {Send Servidor a} end
   {Browse Output}
   % Posible salida: [a#1]|[a#1 b#1]|[a#2 b#1]|[a#2 b#1 c#1]|[a#2 b#1
c#2]|...
end
```